Развој софтвера 2

Дистрибуирани системи и отпорност на отказе

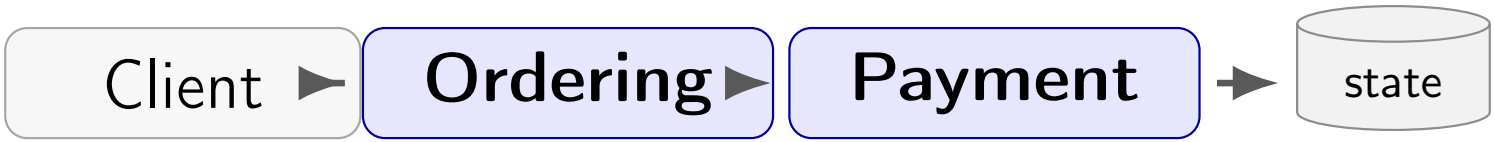
Стефан Капунац

Математички факултет, Универзитет у Београду

школска 2026/27. година

Главна идеја часа

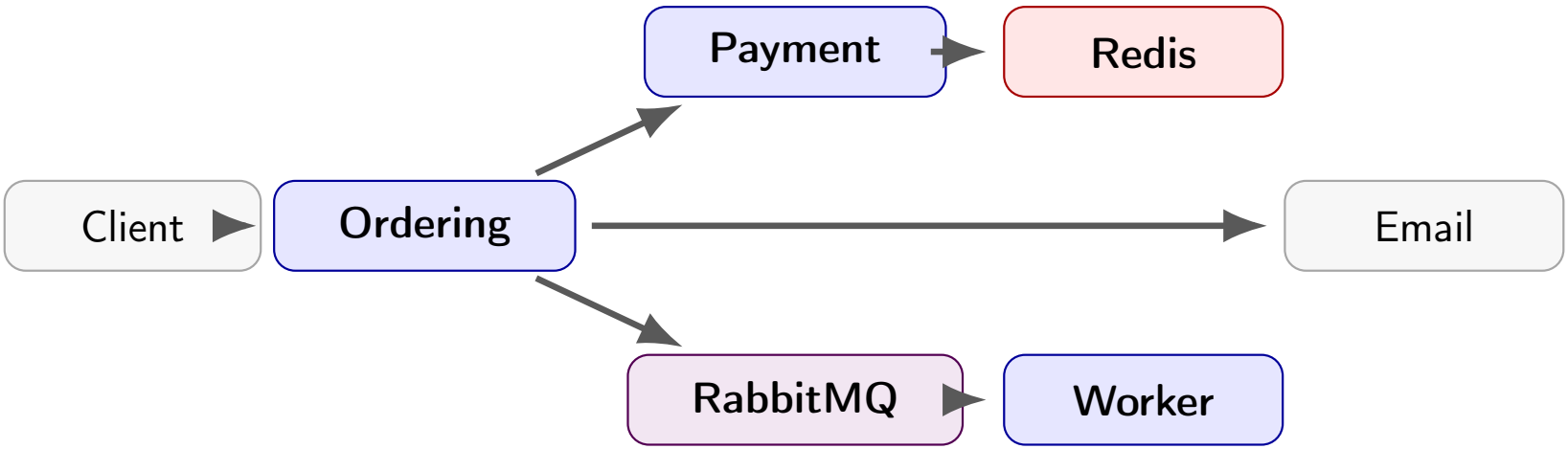
Локални позив методе и мрежни позив нису исто.



Данас нас занима

Шта caller може да зна када dependency касни, падне или одговор не стигне — и како систем пројектујемо да то преживи.

Један demo систем за цео час



- Исти систем ћемо намерно кварити на различите начине.

Ко шта ради у овом систему?

Синхрона грана

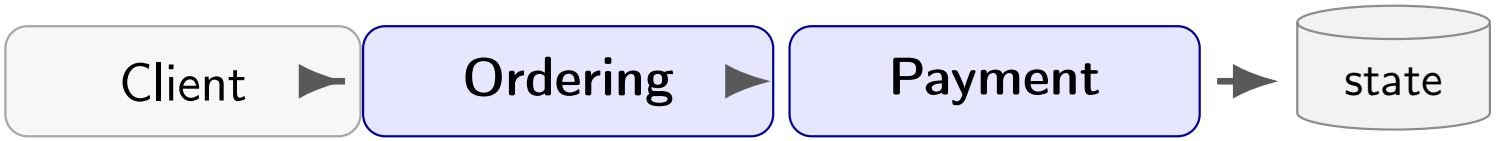
Client шаље поруџбину Ordering-у. Ordering позива Payment преко HTTP-а и чека исход наплате.

Асинхрона грана

Ordering шаље посао за фактуру у RabbitMQ. Worker је засебан позадински процес: преузима посао, прави фактуру и шаље ACK.

- Redis чува demo бројаче и стање за дедупликацију.

Нормалан случај: све делује једноставно



- Ordering шаље захтев Payment-у и добија одговор.
- Овај случај подсећа на локални позив функције — али само док све ради.

Partial failure: један део система може да закаже

Локални процес

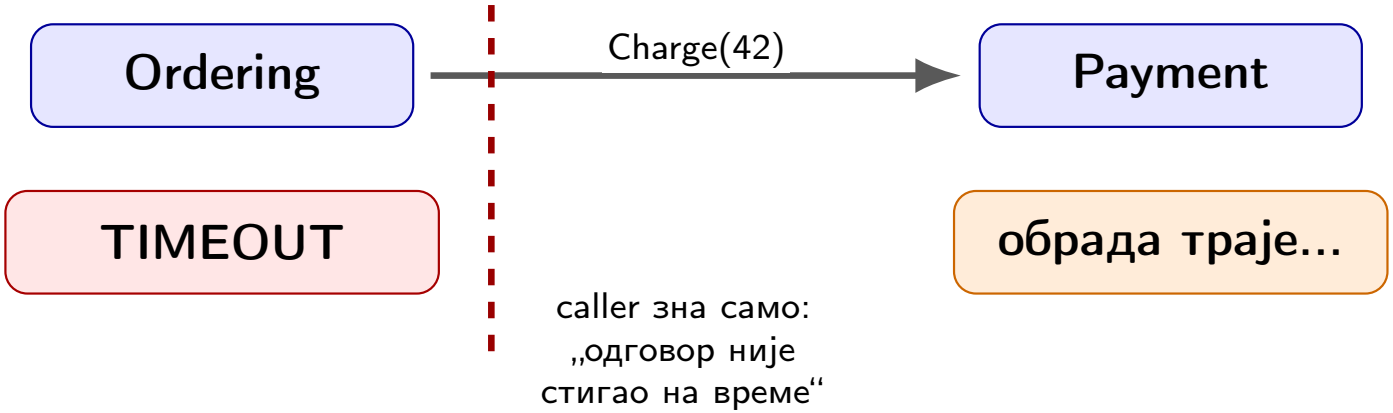
Ако функција ради у истом процесу, простор могућих кварова је релативно мали.

Мрежа

Caller, мрежа и dependency могу да отказују независно једни од других.

- latency може нагло да порасте;
- пакет/веза може да се прекине;
- dependency може да обради захтев, а да caller никада не види одговор.

Timeout: шта заправо знамо?

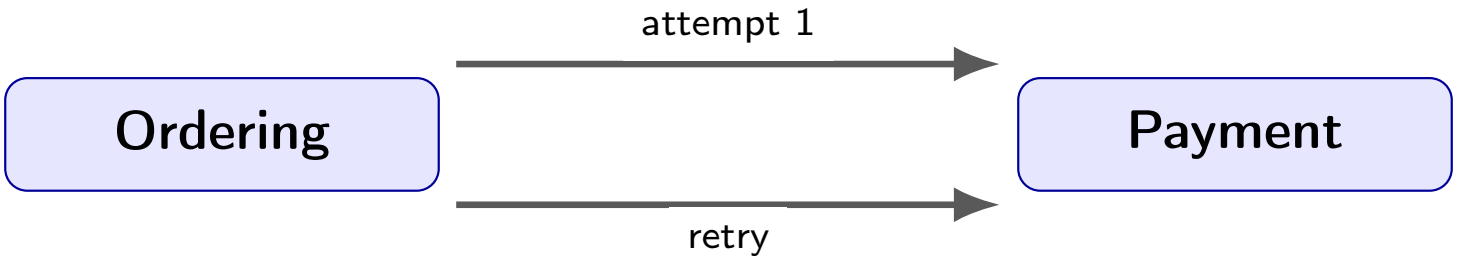


Кључна поента

Timeout не значи „операција није извршена“.

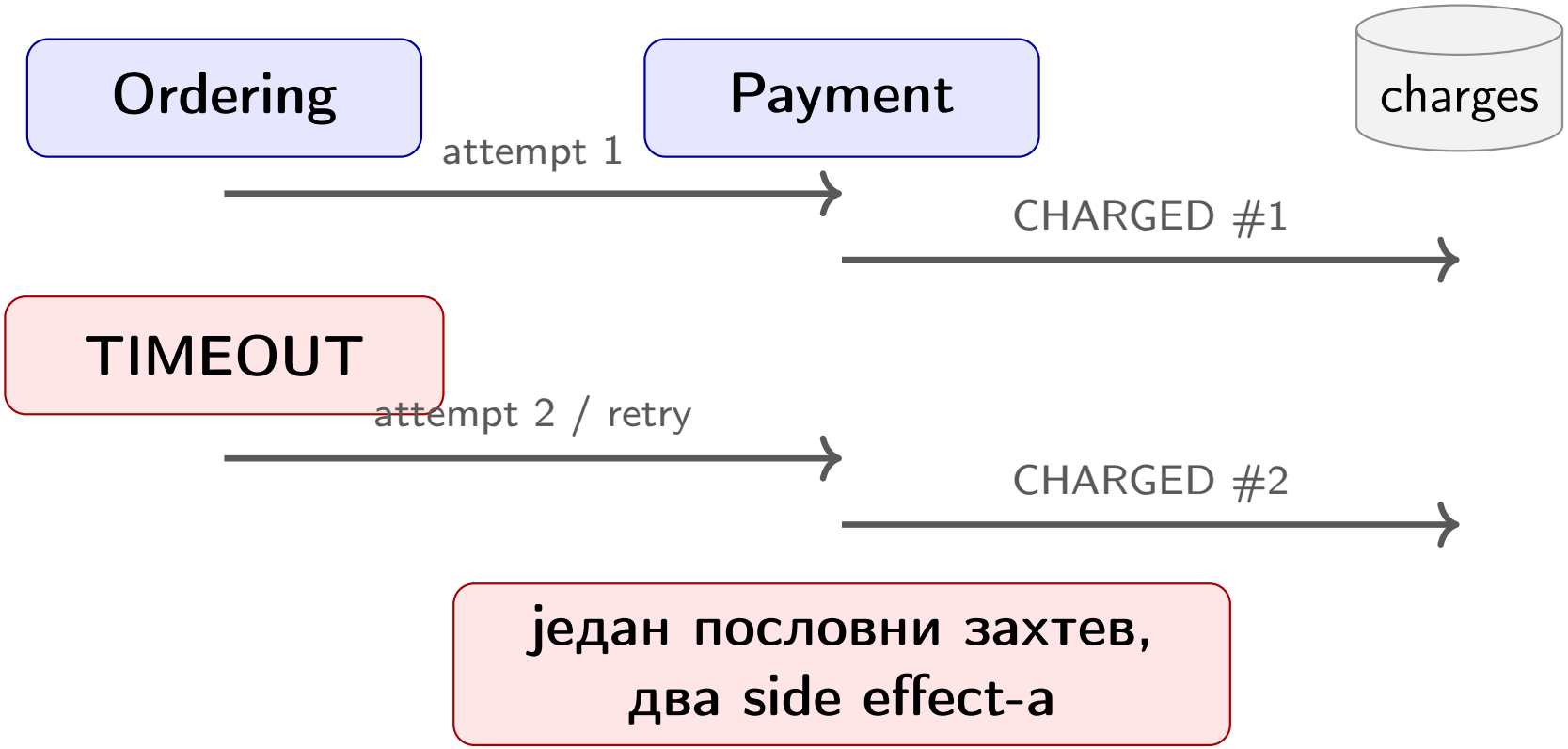
Значи: **одговор није стигао у року који смо спремни да чекамо.**

Retry делује као природно решење...



- Добар је за **привремене** грешке.
- Али retry поново шаље исту пословну намеру.
- Ако је први покушај ипак успео — шта онда?

Retry + side effect = опасност



Идемпотентност

Идеја

Више извршавања **истог пословног захтева** не сме да произведе више side effect-а него једно извршавање.

```
Idempotency-Key: payment:42
```

- Retry може поново послати исти request.
- Сервер препознаје да је то **иста операција**, а не нова наплата.
- Често враћа и резултат првог успешног извршавања.

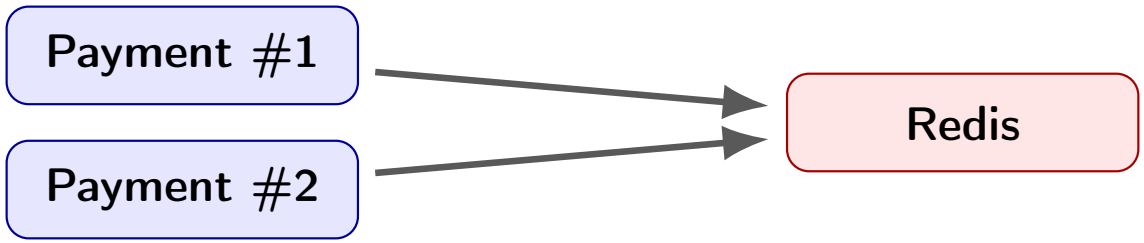
Redis: зашто нам треба заједничко стање?

Локална HashMap

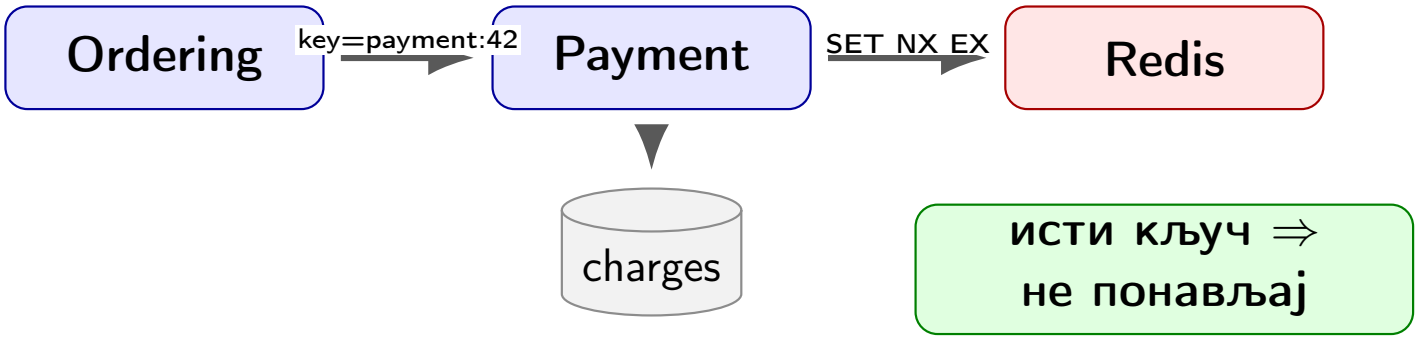
- живи у једном процесу;
- свака инстанца има своју;
- нестаје рестартом процеса.

Redis

- засебан in-memory data store;
- доступан преко мреже;
- више инстанци види исте кључеве;
- TTL и атомичне операције.



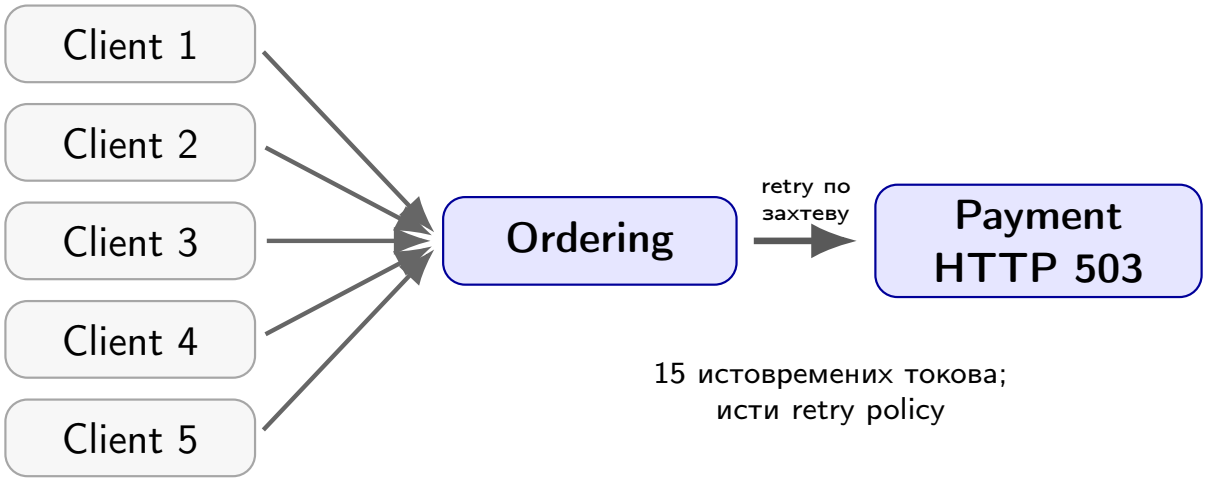
Redis као практична имплементација идемпотентности




```
SET payment:42 IN_PROGRESS NX EX 300
```

- `NX`: упиши само ако кључ још не постоји.
- `EX`: кључ истиче; исти захтев тада може поново проћи.
- После успеха: `COMPLETED + result`.
- Claim, наплата и COMPLETED овде нису једна трансакција.

Retry storm



- Ordering симулира 15 конкурентних захтева и retry-уje позиве ка Payment-у.
- 15 захтева $\times$ 4 покушаја $=$ 60 позива; backoff разређује таласе, jitter их распршује.

Exponential backoff



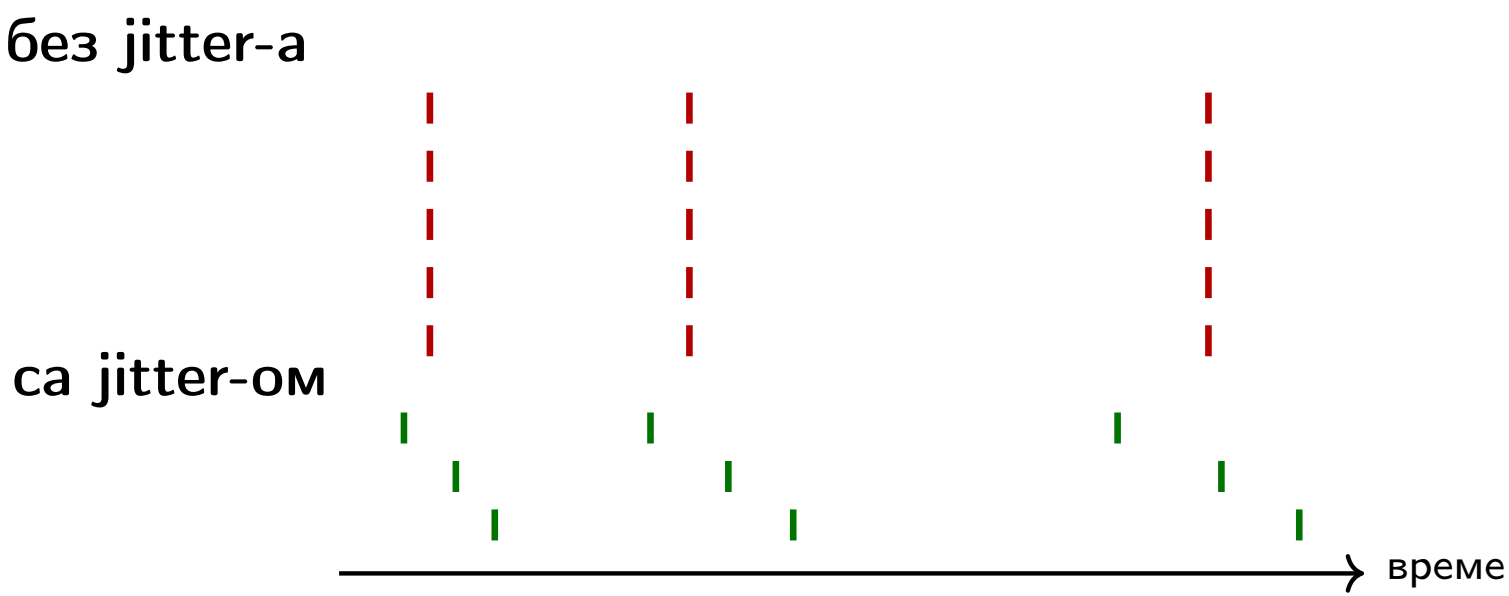
$$\text{delay}_k = \min(\text{cap},\ \text{base} \cdot 2^k)$$

- Не retry-ујемо агресивно док dependency не ради.
- Размак између покушаја расте.

Зашто backoff није довољан?

Проблем

Ако су сви клијенти кренули у исто време, исти deterministic backoff их може задржати **синхронизованим**.



Jitter: намерно разбијамо синхронизацију

$$\texttt{delay} = \texttt{exponential_backoff} \times \texttt{random_factor}$$

- Укупан број покушаја не мора бити мањи.
- Али више не стижу сви у истом тренутку.
- Циљ: избегнути *thundering herd*.

Иста појава ван софтвера

Retry без jitter-a

много клијената
исти тајминг
$\Downarrow$
request spike

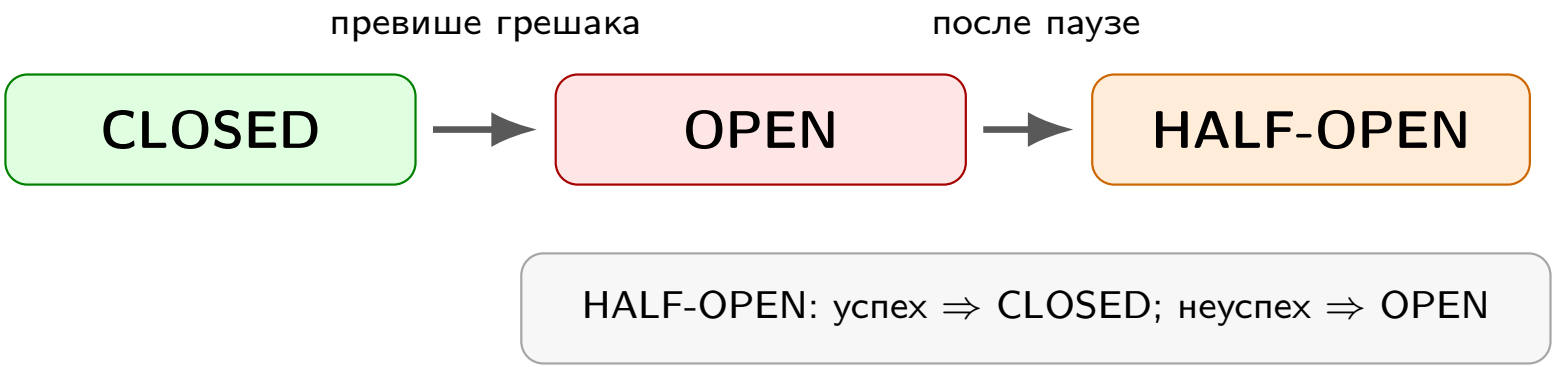
UPS self-test

много уређаја
почетак сваког минута
$\Downarrow$
power-demand spike

Nature Energy, 2026

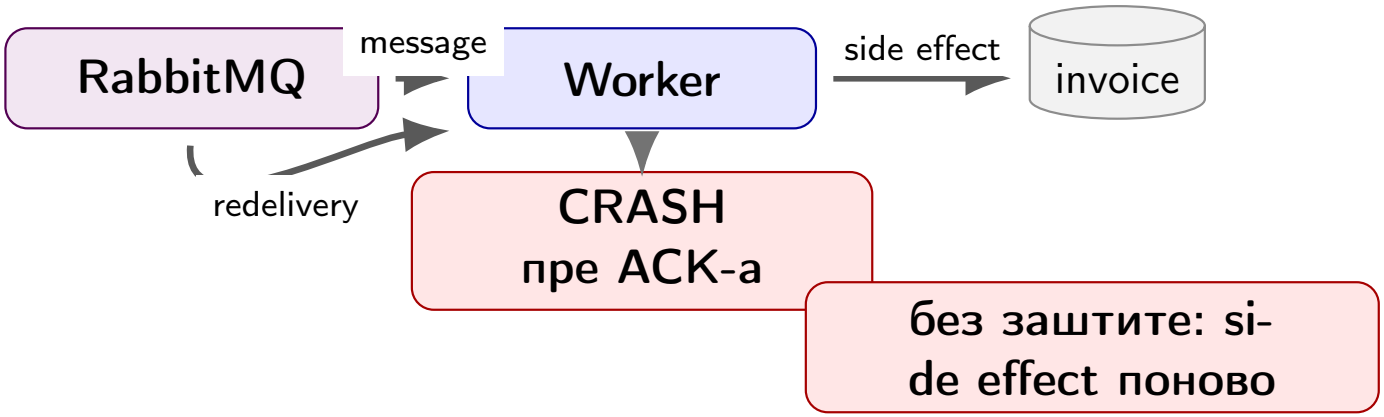
Синхронизовани софтверски self-test-ови UPS/battery система повезани су са 1-минутним осцилацијама у електроенергетским мрежама. Предложено ублажавање укључује randomized scheduling.

Circuit breaker



- **CLOSED**: позиви нормално пролазе.
- **OPEN**: fail-fast — не трошимо ресурсе на dependency који очигледно не ради.
- **HALF-OPEN**: пробни позив да видимо да ли се dependency опоравио.

Нема ACK-а не значи да обрада није извршена



- Worker може да заврши пословни side effect, па да падне **пре ACK-а**.
- Broker види да порука није ACK-ована и може да је поново испоручи.
- Зато consumer често мора бити идемпотентан.

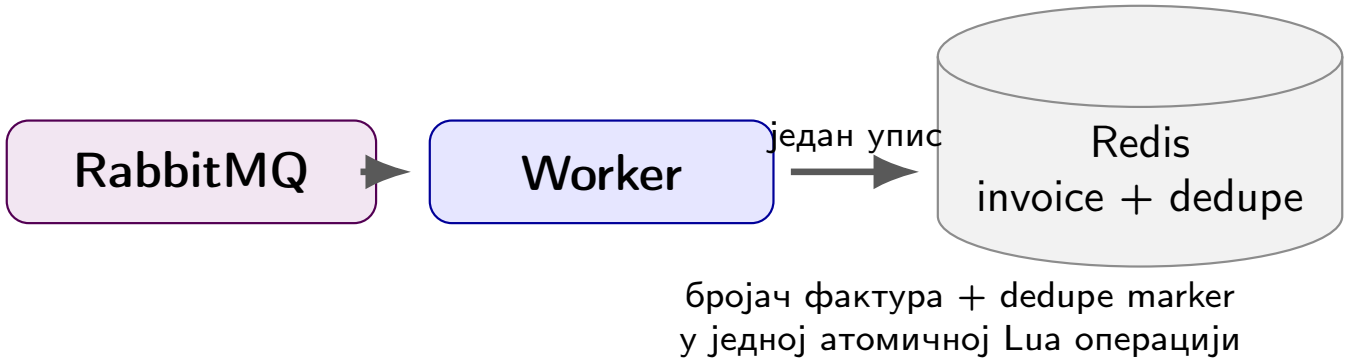
Redelivery: дупликат у пракси

```
invoice-count: 2
```

Питање

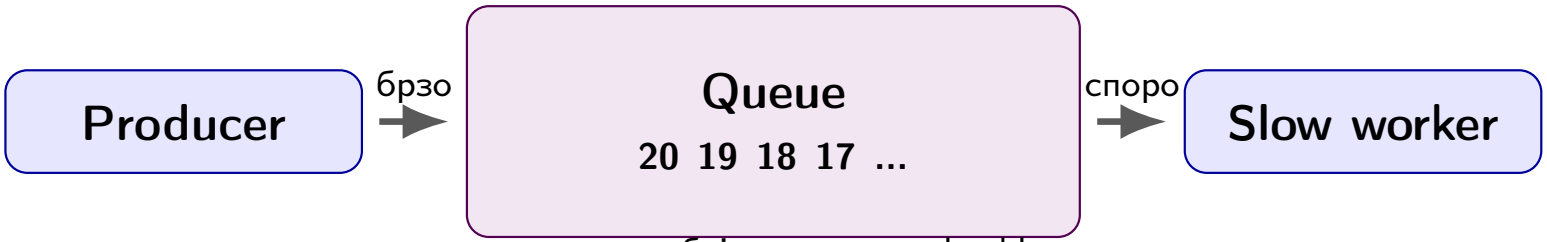
Да ли RabbitMQ може да зна да је наша пословна операција већ извршена?

Идемпотентан consumer



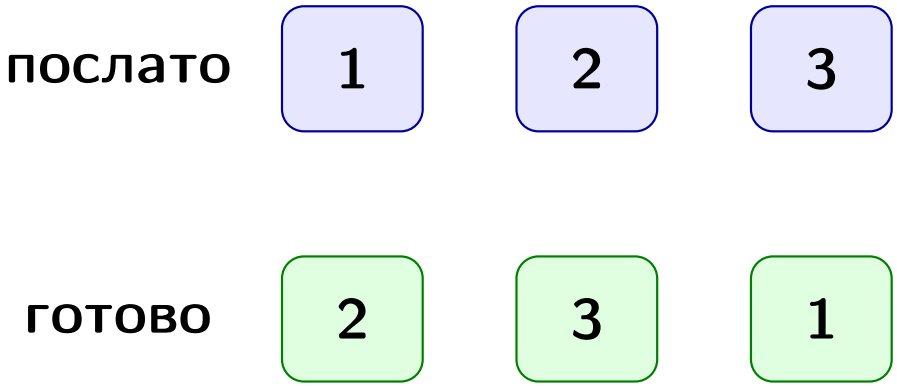
- Message има стабилан `job_id`.
- Dedupe marker и demo side effect уписују се атомично.
- За засебну SQL базу потребна је одговарајућа трансакциона граница.
- Redelivery остаје могућ; овај посао не прави дуплу фактуру.

Burst и backlog; где је backpressure?



- Queue раздваја брзину producer-a и consumer-a.
- Али backlog није бесплатан: расту latency и потрошња ресурса.
- **Backpressure**: downstream сигнализира/намеће да upstream успори.

Ред слања није нужно ред завршетка



- Конкурентна обрада има различита трајања.
- Ако business захтева ред, морамо га **експлицитно** пројектовати.

Rate limiting

Правило

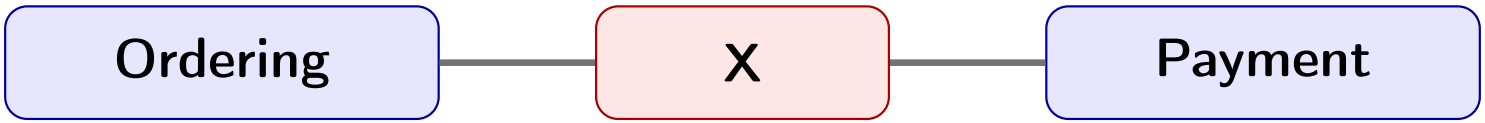
У демоу: укупно 5 захтева у секунди за једну Ordering инстанцу. Лимит је локалан и глобалан за ту инстанцу.

Када пређемо лимит

Одбијамо/успоравамо вишак уместо да пустимо да исцрпи backend.

`HTTP 429 Too Many Requests`

Network partition



- Оба процеса могу бити „жива“, а да не могу да комуницирају.
- Caller често види исте симптоме као код пада dependency-ja: timeout/connect failure.
- Ово је један разлог зашто је partial failure фундаменталан у distributed системима.

Разликовати од HTTP 503

HTTP 503 је примљен одговор сервера. Овај demo не прекида стварну мрежу између сервиса.

HTTP 503 и timeout су различити исходи

HTTP 503

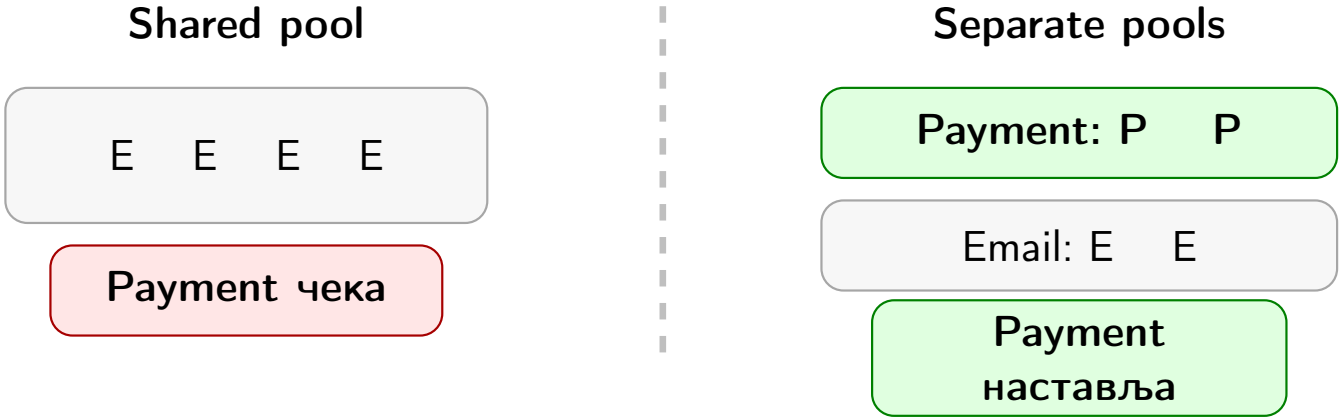
Payment враћа грешку. Ordering добија одговор да услуга није доступна.

Timeout

Ordering није добио одговор у року. Не зна да ли је Payment извршио наплату.

- Circuit breaker може да броји оба као неуспешне позиве.
- То не значи да су узрок и знање caller-а исти.

Bulkhead: изолуј ресурсе



- Спор Email не треба да заузме све слотове и блокира Payment.
- Shared: 4 слота за све. Separate: 2 за Email + 2 за Payment; исти укупан капацитет.
- Изолујемо чекање: Payment не чека да се Email pool ослободи.

Ко имплементира resilience?

Application: business семантика, idempotency key, duplicate policy

Library / framework: retry, timeout, circuit breaker, rate limiter

Proxy / service mesh / cloud: routing, retry policy, timeout, circuit breaking...

Важно

Инфраструктура може да retry-ује HTTP/RPC позив, али не зна да ли је `ChargeCard` безбедно поновити нити који је исправан idempotency key.

Google Cloud пример

- Cloud Service Mesh подржава retry policy и timeout-е.
- Може да примени circuit-breaking ограничења и друге traffic-management политике.
- То смањује количину ручно писаног resilience кода.

Али

Механизам може бити у инфраструктури; **пословна коректност остаје одговорност апликације**.

Шта треба понети са часа?

1. Timeout је **неизвестан исход**, не доказ неуспеха.
2. Retry захтева размишљање о **идемпотентности**.
3. Backoff + jitter штите dependency од retry storm-a.
4. Circuit breaker, bulkhead и rate limiting ограничавају blast radius.
5. Messaging може да ределивeрује; consumer мора бити спреман на дупликате.
6. Queue помаже код burst-a, али backlog и редослед морају бити дизајнирани.

Следећи проблем

Payment је успешно наплатио картицу.

Процес падне пре него што Ordering упише:

`PAID`

Како одржати конзистентност када једна пословна операција прелази више сервиса и база?

Следећи час: подаци, конзистентност и дистрибуиране трансакције.

Извори за даље читање

- Redis документација: `SET` са `NX`/`EX` опцијама.
- RabbitMQ: Consumer acknowledgements, redelivery и reliability.
- Google Cloud Service Mesh: retries, timeouts и circuit breaking.
- AWS Architecture Blog: *Exponential Backoff And Jitter*.
- Nature Energy (30.10.2026): *One-minute frequency disturbances from unintended synchronization in cyber-physical power systems*.